20/7/26

Ex.No:1

Ex:1

STUDY OF VARIOUS NETWORK COMMANDS USED IN LINUX AND WINDOWS.

AIM:-

To study the various network commands used in Linux and windows.

Basic Networking Commands:-

1] arp-a :-

* It is short form of address resolution protocol.
* It shows the IP address of the computer and MAC address of router.

Output:-

_gateway (10.0.2.2) at 52:55:0a:00:02:02 [ether] on enp0s3.

2. hostname :

* It is simplest of TCP/IP commands which display the name of your computer.

output:-

fedora.

3. ipconfig /all :-

This shows the detailed configuration information about your TCP/IP connection including Router, Gateway, DNS, DHCP and type of Ethernet Adapter.

output :-

Host Name : HDC04222b8.

physical address : 88-AE-DD-15-EE-12.

Node Type : Hybrid.

DHCP enabled : No

4. nbstat-a :-

This helps to solve problems with Net BIOS name resolution.

output :-

NBTSTAT [[-a RemoteName] [-A IP address] [-c] [-n] [-r] [-R] [-RR] [-s] [-S] [interval]]

-a (adapter status) → Name

-A (Adapter status) → IP addresses.

-c (cache)

-n (names)

-r (resolved)

-R (Reload)

5. netstat :-

It displays a variety of statistics about a computers active network TCP/IP connections.

op :-

Proto	Local Address	Foreign Address	state
TCP	172.16.10.193:4495	DESKTOP-8M7G1G9:57950	Established
TCP	172.16.10.193:7680	172.16.8.161:64132	TIME_WAIT.

6. nslookup:

It is a tool used to perform DNS lookups in Linux. It is used to display DNS details such as IP address of a particular computer.

output :-

Defaut server: Unknown

Address : 172.16.8.1

7. pathping :-

It is unique to window's and is combination of ping and Tracert command

o/p :-

pathping [-g host-list] [-h maximum_hops] [i address] [-n][-p period] [-q num_queries] [-w timeout] [-4][-6] target_name.

Some Important Linux Commands

1] ip address show

To show the IP addresses assigned to an interface on your server.

o/p:-

inet 127.0.0.1/8 scope host lo

inet6 ::1/128 scope host noprefixroute.

2] ip address add 192.168.1.254/24 dev enps03

To assign an IP to an interface.

for eg: enps03.

o/p:-

2. enp0s3: <BROADCAST, MULTICAST, UP, LOWER_UP>...

inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3

inet 192.168.1.254/24 scope global secondary enp0s3.

3] ip address del 192.168.1.254/24 dev enps03.

To delete an IP on an Interface.

inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3.

4. ip link set eth0 up

After the status of the interface by bringing the interface eth0.

O/P:-

2: enp0s3: <BROADCAST, MULTICAST, UP, LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000.

5. ip link set eth0 down.

To delete alter the status of the interface by bringing the interface eth0.

O/P:- 2: enp0s3: <BROADCAST, MULTICAST> mtu 1500 qdisc fq_codel state DOWN mode DEFAULT group default qlen 1000.

6. ip link set enp0s3 promisc on

After the status of the interface by enabling promiscuous mode for:

O/P:- <BROADCAST, MULTICAST, PROMISC> mtu 1500 qdisc fq_codel stat link/ DOWN mode DEFAULT group default qlen 1000.

STUDY OF NETWORK CABLES

AIM:-

To study the different types of Network Cables.

Cable type	Category	Max Data Transmission.	Advantages/ Disadvantages	Use.
UTP	Category 3	10bps	Advantages:- * Cheaper * Easy to install. Disadvantages. More prone to EMI.	Fast ethernet. 10 Base T Ethernet.
	Category 5	Upto 100 Mbps		
	Category 5e	1 Gbps.		
STP	Category 6, 6a	10Gbps.	* Shielded * Faster than UTP * Less noise	* Gigabi Ethernet. * Data centres
SSTP	Category 7	10Gbps	Dis * Expensive. * Greater installation effort.	* Gigabi Ethernet
Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps.	* high bandwidth. * Low loss bandwidth. * versatile. Dis * cost * size is bulky	Speed of signal is soom TV network high speed internet Conne

Fibre optics cable.

Single Mode
Multi mode.

100Gbps.

Adv
* High speed
* High security

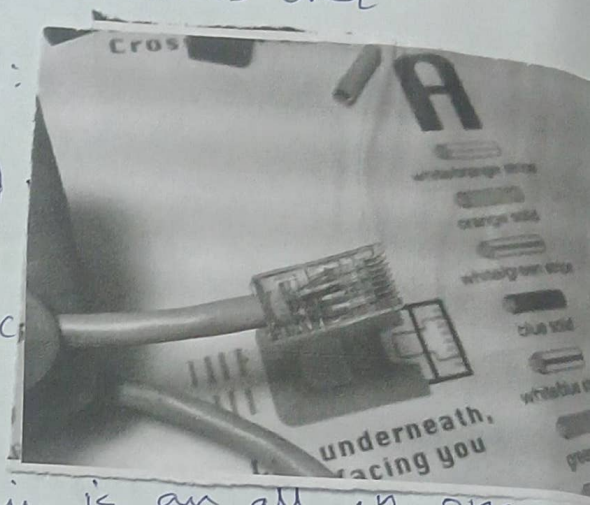
dis.
* Expensive
* Require skilled installer.

Max distance of f.o cable is around 100 metres.

b] Make your own Ethernet Cross over cable / straight cable:

Tools and parts needed:

* Ethernet cabling . Cat5e gigabit support.
* A crimping tool. This is an all in one networking tool shaped to push down the pins in the plug.
* Two RJ45 plugs.
* optional two plug shields.



Step 1:- To start construction of the device, begin by threading shields onto the table.

Step 2:- Next, strip approximately 1.5cm of cable shielding from both ends. The crimping tool has a round area to complete this task.

PACKET TRACER.

AIM:-

To study the Packet traser tool Installation and User Interface Overview.

Introduction :-

1. It allows you to model complex systems without need for dedicated equipment.

2. It helps you to practice your network configuration and troubleshooting skills via computer or an Android or IOS based mobile device.

3. It is available for both the linux and window desktop environments.

4. Protocols in Packet Traser are coded to work and behave in the same way as they would on real hardware.

3. Logical / physical workspace tabs - These tabs allow you to toggle between the logical and physical work areas.

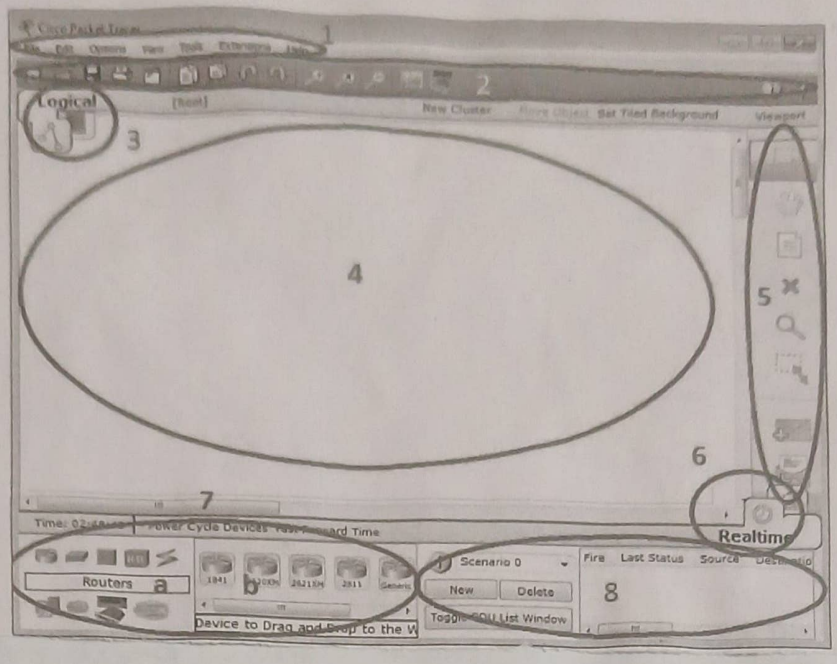
4. Workspace - This is the area where topologies are created and simulations are displayed.

5. Common tools bar - This toolbar provides controls for manipulating topologies such as select, move layout, place note, delete, inspect, resize shape and add simple / complex PDU.

6. Real-time / simulation tabs - These tabs are used to toggle b/w real and simulation ~~tabs~~ modes.

7. Network component box - This component contains all of the network and end devices available with packet Tracer.

8. User created packet Box - Users can create highly customized packets to test their topology from this area.



d) Analyse the behaviour of network devices using CISCO PACKET TRACER simulator.

1. From the network component box, click and drag-and-drop the below components:

   a. 4 Generic PC's and one HUB.

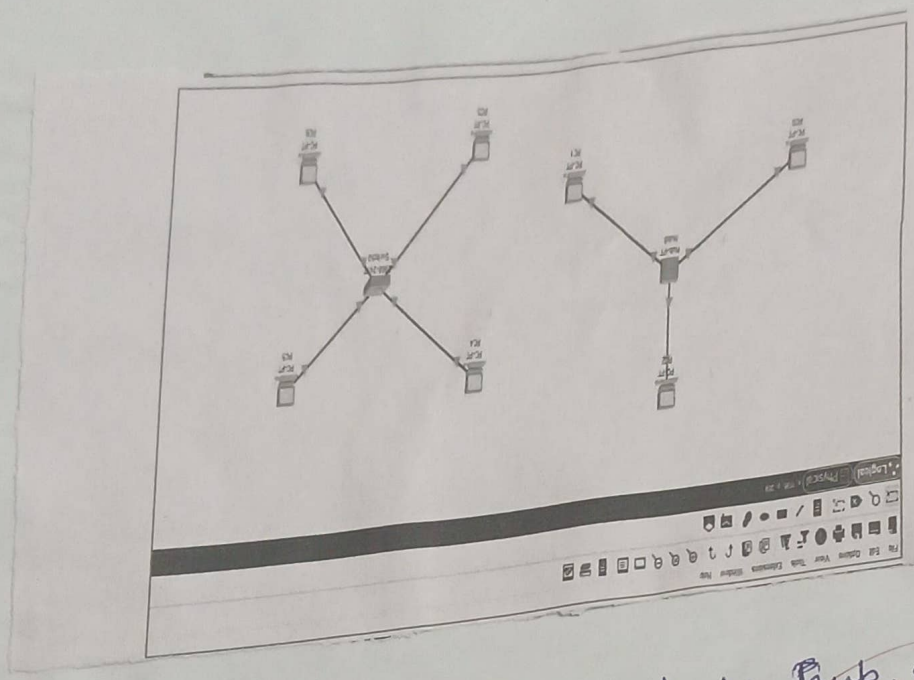
   b. 4 Generic PC's and one Switch.

2. Click on Connections:

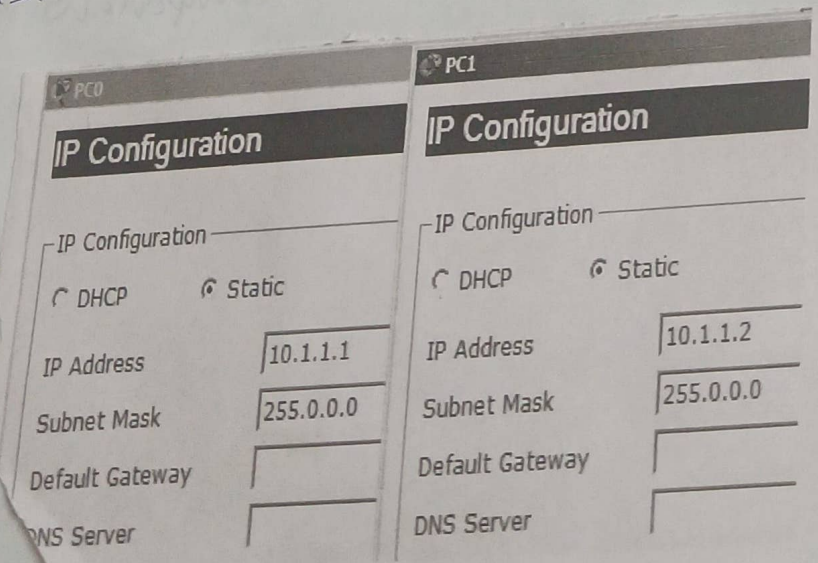
   a. click on Copper ~~straight~~ - Through Cable.

   b. select ~~one of~~ the PC and Connect it to HUB using the cable. ~~This~~ The link LED should glow in green, indicating that the link is up. Similarly connect remaining ~~3~~ PCs ~~to~~ HUB.

c. Similarly connect 4PCs to the switch using copper straight through cable.



3. Click on the PC's connected to Hub, go to Desktop tab, click on IP Configuration, and enter an IP address and subnet mask.



Click on the PDU from common tool bar.

a. Drag and drop it on one of PC.

4. Observe the flow of PDU from Source PC to destination PC by selecting the Real time mode of simulation.

5. Repeat step #3 to step #5 for the PC's connected to the switch.

6. Observe how HUB and switch are forwarding the PDU and write your observation and conclusion about the behaviors of switch and HUB.

Result:

Thus experiment for CISCO platter Network is done using Simulation Tool.

Ex.No:4 LOCAL AREA NETWORK.

AIM:-

To setup and configure a LAN using a switch and Ethernet cables in your lab.

What is a LAN?

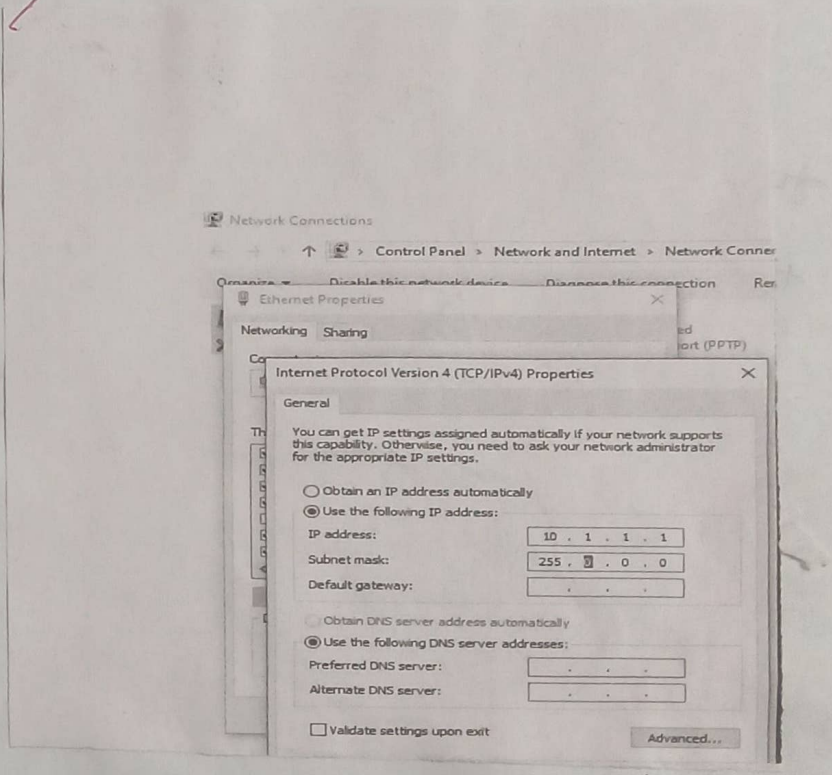
Step 1:- plan and Design an appropriate network topology taking into account network requirements and equipment location.

step 2:- You can take a Computer, a switch with 8, 16, 24 ports which is sufficient for network of these sizes and 4 Ethernet cables.

Step 3:- connect your Computers to network switch via an Ethernet cable, which is as simple as plugging one end of the Ethernet cable into your computer.

Step4 :- Assign Ip address to your PCs

1. Log on to the client computer as owner
2. click network and Internet Connections
3. Right click LAN / Ethernet → Go to Prop → Select Internet Protocol → click on Properties → Select use the following IP address option and assign ipaddress.



Step5 :- Configure a network switch :

1. Connect your computer to the switch To access the switch's web interface, you need to connect your computer to the switch using an Ethernet cable.

2. Assign Ip address as : 10.1.1.5.
Subnet mask 255.0.0.0.

Step 6:- Check the connectivity b/w switch and other machine by using ping command in the command prompt of the device.

Step 7:- Select a folder → go to Prop→ Click sharing tab → share it with everyone on the same LAN.

Step 8:- Try to access the shared folder from others computers of the network.

Result:-

Thus the setup of Local Area Network using a switch and Ethernet is executed.

Ex.No:5

Capture Tool: Wireshark.

Aim:-

Experiments on Packet capture tool: Wireshark.

Packet sniffer:-

* sniffs messages being sent / received from / by your computer.

* stores and display the contents of various protocol fields in the messages.

* passive program

→ never send packets itself.

→ no packet addressed to it.

→ Receives a copy of all packets.

Packet sniffer structure diagnostic tools:-

Tcpdump

Eg:- tcpdump -enr host 10.129.41.8 -w exe.out.

Wireshark

Wireshark -r exe3.out

Description:-

Wireshark:-

A network analysis tool formerly known as Ethereal, captures packet in real-time and display them in human readable format. wireshark includes filters, color coding, other features that let you dig deep into network traffic and inspect individual packets.

What we can do with wireshark:

* Capture network traffic
* Decode packet protocols using dissectors
* Watch smart statistics
* Analyze problems.

Wireshark used for:-

* Network administrators: Trouble shoot network problem.
* Network security engineers: examine security problems.
* Developers: debug protocol implementation.

Getting wireshark:-

wireshark can be downloaded for windows or macOS from its official website. for linux or another unix like system, wireshark will be found in its package repositories.

Capturing Packets:-

After downloading and installing wireshark, launch it and double click the name of a network interface under capture to start capturing packets.

The "Packet list" Pane

This displays all the packets in the current capture file. Each line in the packet list corresponds to one packet in the capture file. If you select a line in this pane, more details will display.

The "Packet Details" Pane

This shows the current packet in a more detailed form. This shows the protocols and fiels of packet selected in the Packet list pane.

The "Packet Bytes" pane

This shows the Data of the Current Packet in a hexdump style.

Color Coding:-

You'll probably see packets highlighted in a variety of different colors. It uses colors to help you identify the types of traffic at a glance; By default, light purple is TCP traffic, light purple is UDP and blacks is for Packets with errors.

To view exactly what color codes mean, click View > Coloring rules. You can also customize and modify coloring rules from here.

Sample Captures:-

The wiki contains a page of sample capture files that you can load and inspect. Click File > Open in Wireshark and browser for your downloaded file to open one.

Filtering packets:-

Still, you'll likely have a large amount of packets to sift through. That's where wireshark's filters come in.

Inspecting packets:-

click a packet to select it and you can dig down to view its details.

Capturing and analysing packets using Wireshark tool:-

To filter, capture, view, packets in Wireshark Tool.

Capture 100 packets from Ethernet IEEE 802.3 LAN interface and save it.

Procedure :-

1. Select Local Area connection in wireshark
2. Go to capture option.
3. select stop capture automatically after 100 packets.
4. Then click Start capture.
5. Save the packets.

1. Create a file to display only TCP/UDP Packets inspect the packets and provide the flow graph.

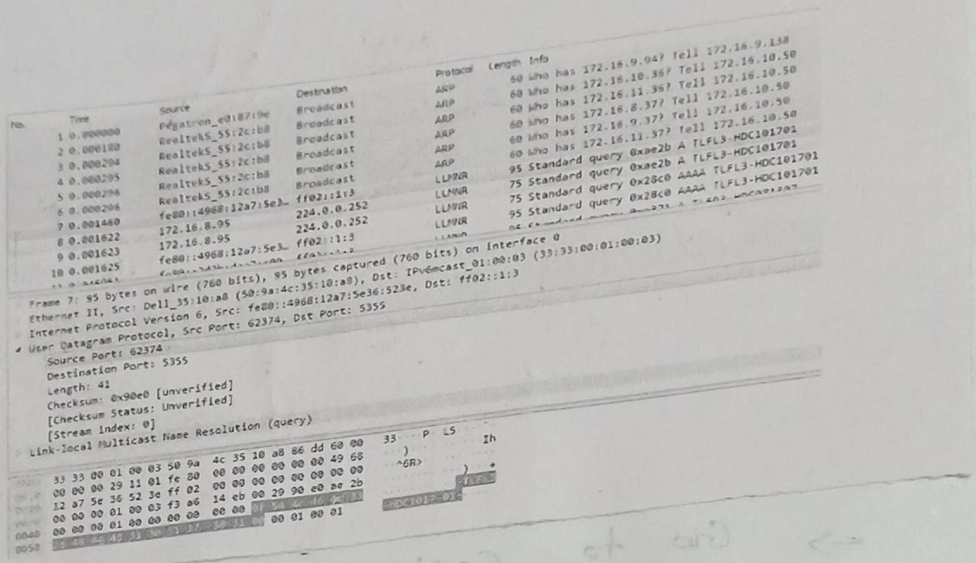
→ 1. Select Local Area Connection in Wireshark
→ Go to Capture option.
→ Select stop capture automatically after 100packet
→ Then click Start capture.
→ search TCP packets in search bar.
→ To see flow graph click Statistics Flow graph.
→ save the Packets.

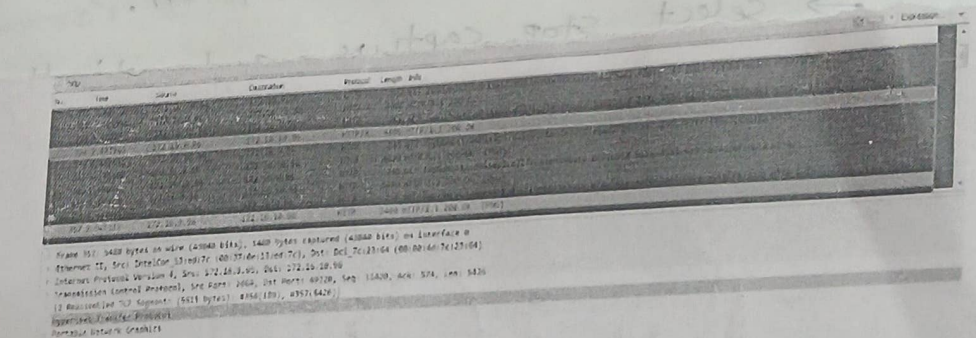
2. create a filter to display only ARP packets and inspect the packets. procedure

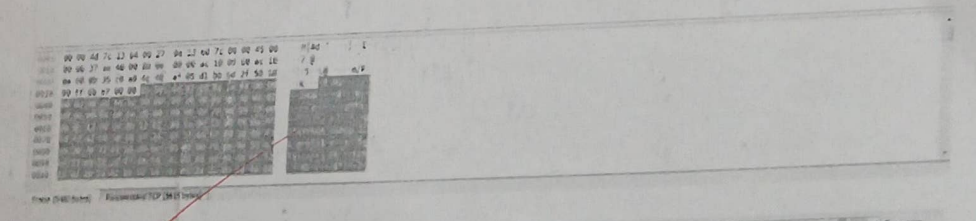
→ search ARP packets in search bar.

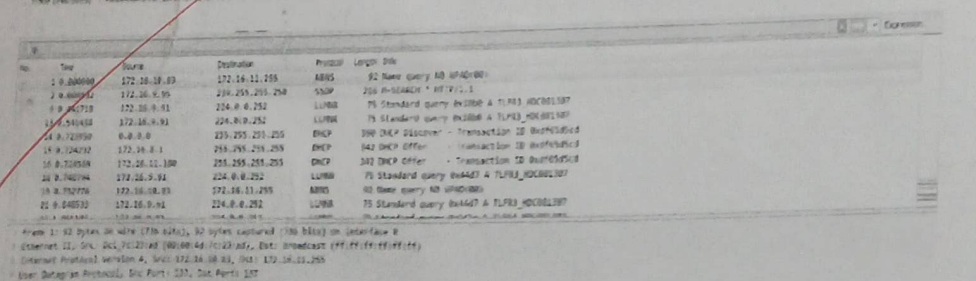
3. create a filter to display only DNS packets and provide the flow graph.

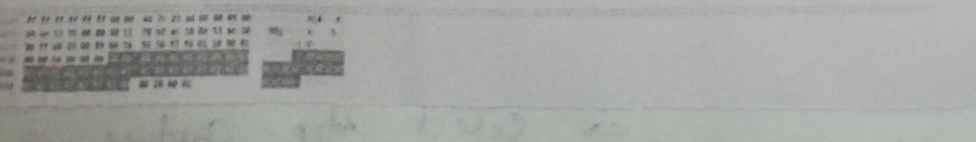
→ Go to capture option.


No. Time Source Destination Protocol Length Info
1 0.000000 Pegatron_e0:87:9c Broadcast ARP 60 Who has 172.16.9.94? Tell 172.16.9.138
2 0.000180 RealtekS_55:2c:b8 Broadcast ARP 60 Who has 172.16.10.36? Tell 172.16.10.50
3 0.000204 RealtekS_55:2c:b8 Broadcast ARP 60 Who has 172.16.11.36? Tell 172.16.10.50
4 0.000295 RealtekS_55:2c:b8 Broadcast ARP 60 Who has 172.16.8.37? Tell 172.16.10.50
5 0.000296 RealtekS_55:2c:b8 Broadcast ARP 60 Who has 172.16.9.37? Tell 172.16.10.50
6 0.000296 fe80::4968:12a7:5e3… ff02::1:ff3 LLMNR 75 Standard query 0x28c0 AAAA TLFL3-HDC101701
7 0.001460 172.16.8.95 224.0.0.252 LLMNR 75 Standard query 0x28c0 AAAA TLFL3-HDC101701
8 0.001822 172.16.8.95 224.0.0.252 LLMNR 95 Standard query 0xae2b A TLFL3-HDC101701
9 0.001623 fe80::4968:12a7:5e3… ff02::1:3 LLMNR 95 Standard query 0xae2b A TLFL3-HDC101701
10 0.001825
Frame 7: 95 bytes on wire (760 bits), 95 bytes captured (760 bits) on interface 0
Ethernet II, Src: Dell_35:10:a8 (50:9a:4c:35:10:a8), Dst: IPv6mcast_01:00:03 (33:33:00:01:00:03)
Internet Protocol Version 6, Src: fe80::4968:12a7:5e36:523e, Dst: ff02::1:3
User Datagram Protocol, Src Port: 62374, Dst Port: 5355
Source Port: 62374
Destination Port: 5355
Length: 41
Checksum: 0x90e0 [unverified]
[Checksum Status: Unverified]
[Stream Index: 0]
Link-local Multicast Name Resolution (query)
33 33 00 01 00 03 50 9a 4c 35 10 a8 86 dd 60 00
00 00 00 29 11 01 fe 80 00 00 00 00 00 00 49 68
12 a7 5e 36 52 3e ff 02 00 00 00 00 00 00 00 00
00 00 00 01 00 03 f3 a6 14 eb 00 29 90 e0 ae 2b
00 01 00 01

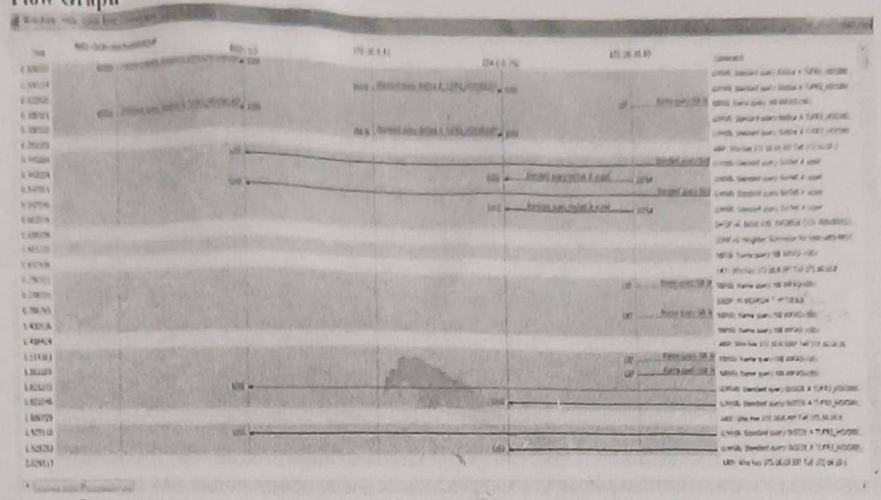








Flow Graph

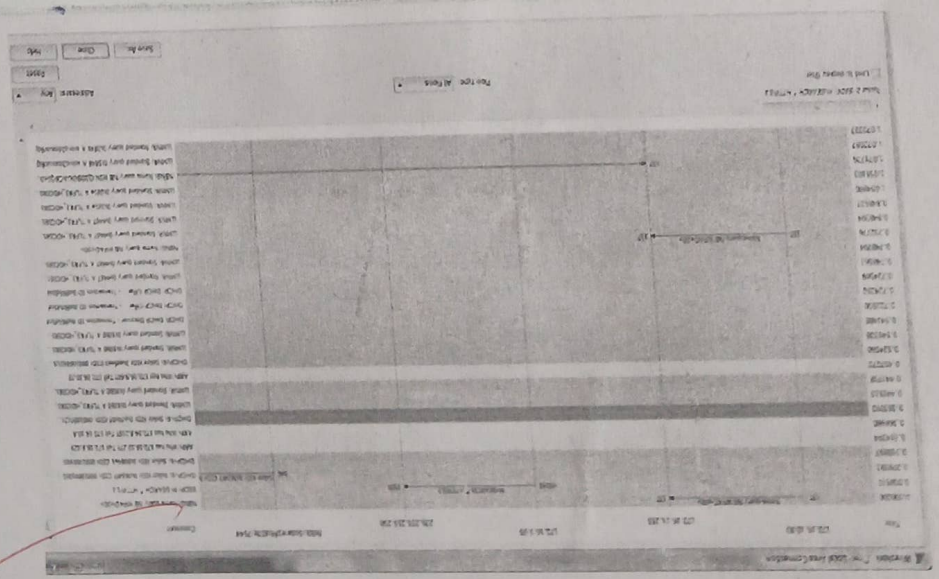


arp

No.	Time	Source	Destination	Protocol	Length	Info
8	0.255305	Foxconn_c9:c5:f0	Broadcast	ARP	60	Who has 172.16.10.15? Tell 172.16.10.3
14	0.692936	Foxconn_d0:ac:46	Broadcast	ARP	60	Who has 172.16.8.39? Tell 172.16.10.8
19	1.418424	Foxconn_c9:c9:91	Broadcast	ARP	60	Who has 172.16.8.106? Tell 172.16.10.26
24	1.880729	Foxconn_d0:ac:46	Broadcast	ARP	60	Who has 172.16.8.40? Tell 172.16.10.8
27	2.029517	Giga-Byt_92:d2:ef	Broadcast	ARP	60	Who has 172.16.10.33? Tell 172.16.10.1
41	2.509905	Giga-Byt_7c:c5:34	Broadcast	ARP	60	Who has 172.16.9.82? Tell 172.16.9.111
44	2.602358	Foxconn_c9:c8:24	Broadcast	ARP	60	Who has 172.16.8.139? Tell 172.16.10.22
46	2.743021	Dell_35:11:11	Broadcast	ARP	60	Who has 172.16.8.118? Tell 172.16.10.195
56	3.201822	Giga-Byt_92:d2:ef	Broadcast	ARP	60	Who has 172.16.10.34? Tell 172.16.10.1
60	3.237061	Giga-Byt_7c:c5:34	Broadcast	ARP	60	Who has 172.16.9.82? Tell 172.16.9.111

Frame 119: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0
Ethernet II, Src: IntelCor_13:ed:7c (00:27:0e:13:ed:7c), Dst: RealtekS_b2:60:90 (00:e0:4c:b2:60:90)
Address Resolution Protocol (reply)

```
0000  00 e0 4c b2 60 90 00 27  0e 13 ed 7c 08 06 00 01   ..L.`..'  ...|....
0010  08 00 06 04 00 02 00 27  0e 13 ed 7c [redacted]
0020  00 e0 4c b2 60 90 ac 10  09 6a                      ..L.`...  .j
```



Result :- A capture tool using wireshark

Ex.No:6

Error Correction in Data Link layer.

AIM:-

Write a program to implement error detection and correction using Hamming code concept. Make a test run to input data stream and verify error correction feature.

Error Correction at Data Link Layer:-

Hamming code is a set of error correction codes that can be used to detect and correct the errors that can occur when the data is transmitted from the sender to receiver. It is a technique developed by R.W. Hamming for error correction.

Code (sender.py)

```
data= list(map(int, input("Enter data bits: ").
strip()))
m = len(data)
r=0
while (2**r)< (m+r+1)
    r+=1
n= m+r
hamming=[0]*(n+1)
j=0
for(i in range (1,n+1):
    if( 1*(i-1))!=0:
        hamming[i]=data[j]
```

```
for i in range (r):
    pos = 2 ** p
    parity = 0
for i in range (1, r+1):
    if i pos
    parity = hamming[i]
    hamming[pos] = parity
    code = hamming[i]
    print ("Databits : data)
    print ("hamming code : code)
```

Output:-

Enter data bits = 1011

Data bits [1, 0, 1, 1]

Hamming code : [0, 1, 1, 0, 0, 1, 1]

Receive code (receiver.py)

```
received = list (map (int, input ("Enter
received hamming code : "). strip()))
    n = len (received)
    r = 0
    while (2 ** r)
        r += 1
    error pos = 0
    for p in range (n):
        pos = 2 ** p
        parity = 0
    for i in range (1, n+1):
        if i in pos:
            parity ^= received (i - 1)
```

```
if parity != 0:
    error_pos += pos
if error_pos == 0:
    print("No error detected")
else:
    print("error detection at position:", error_pos)
    received[error_pos-1] ^= 1
    print("Corrected hamming code:", received)
data[]
for i in range(1, n+1):
    if (i, (i-1)) != 0:
        data.append(received[i-1])
print("original data list:", data)
```

output:-

Suppose tic. 01100 11 change to 01111 01

Error detected at position: 4

Corrected hamming code: [0, 1, 1, 0, 0, 1, 1]

original data bits: [1, 0, 1, 1]

Result:-

Thus, the hamming code algorithm was successfully implemented in python, the sender generated the hamming code from the given data bits. The receiver successfully detected and corrected a single Bit errors.

Ex.No:7

Sliding Window Protocol

Aim:-

To write a program to implement flow control at data link layer using sliding window protocol simulate the flow of frames from one node to another.

Sender code:-

```
import time
import os
Print ("=" * 55)
Print ("sliding window protocol - sender")
Window size = int(input ("Enter window size"))
if (window_size <=0):
    Print ("Invalid window size!")
    exit()
    message = input ("Enter text msg:")
if not message:
    Print ("Message cannot be empty!")
    exit()
    frames = []
for i, character in enumerate (message):
    frames.append ({"frames.no":4,
    "data": character
    })
Print ("Frames created:")
```

```
print( frame No: {frame['frame_no']}
Data : {frame['data']}')
total frames = len(frames)
base = 0
while base < label frames
    end = min(base + windowsize, label_frames)
    print("sending frames {base} to find {}
    with open("sender_buffer", w) as file:
    file i in range {base  end()
try:
    with open("receiver_Buffer:", "r") as file:
        response = file.readline().strip()
except FileNotFoundError:
    print("receiver_buffer not found")
parts = response.split()
if (len(parts) != 2:
    print("Invalid response received")
    time.sleep(1)
print("All frames transmitted Successfully")
print("original message {message}")
```

Output:-

Sliding window Protocol Sender

Enter window size: 3

Enter test message: HELLO.

Sender Buffer

Frame No : 0 DATA : 4

Frame No : 1 DATA : E

Frame No : 2 DATA : L

Receiver response : ACK 3

all frames transmitted successfully

original message : HELLO.

Receiver Code :-

```
Import os
sender_file = "sender_buffer.txt"
receiver_file = "receiver_buffer.txt"
expected frame = 0
while True:
    if not os.path.exist (sender_file);
    print ("\n frames received")
    error = false
    error_frame = None
else
    with.open (RECEIVER_file,w")as file:
        file.write (f "ACK {expected file}")
    print (\n") \n ALL frames received Correctly)
    print (f"ACK: {expected_frame}")
```

OUTPUT:-

frames received.

frame No : 0 Data : H

frame No : 1 Data : E

frame No : 2 Data : L

All frames received.

Result:-

Thus, the sliding window protocol was implemented successfully using Python and, flow of frames b/w sender and receiver was simulated using ACK/NACK.

Ex. No: 8

NMAP TO DISCOVER LIVE HOSTS USING NMAP SCANS

AIM:-

To discover live hosts on a network using Nmap host-discovery scans - ARP scan, ICMP scan, and TCP / UDP ping scan - and to understand which protocol operates at which layer of TCP/IP model, when each one applies.

Nmap:-

It is a open source network scanning tool created by Gordon Lyon and first released in 1997.

Procedure:-

1] Log into Try Hackme and open the Nmap live Host Discovery room.

2] Work through tasks 1-3 and answer the simulator based question.

3] Deploy the lab network in Task 4 and start the attackbox note target subnet.

4] Run in ARP scan. PR < subnet >/24 and record how many host respond.

5] Ru...

6) Run ICMP timestamp and ICMP address.

7) Run TCN SYN(PS) TCP ACK, UDP scan

8) optionally capture the scan traffic using TopDump on wireshark on tuno interface.

Result:-

This Nmap host discovery scan were Performed successfully using TCP, SYN, FCP, ACK UDP ping technique. the scan result network traffic were observed.

Ex. No: 9

Implementation of subnetting in cisco packet Tracer Simulator.

Aim:-

To implement subnetting in cisco Packet Tracer by creating multiple subnets, assigning static IP addresses connecting the subnets using router, testing packet transmission.

Theory:-

Subnetting is the process of dividing one network into smaller networks called subnet. It helps in efficient use of IP's addresses and allow multiple network to be created from a single network address.

Requirements:-

Cisco Packet Tracer

Router

Switches

PCs

Copper straight through cables.

Procedure:-

1. Open cisco packet Tracer and create a new network topology.

3] Connect the PC's to switch to the router using suitable cables.

4] Divide the network 192.168.1.0/24 into smaller subnet using 127 subnet mask.

5] Assign static IP address to all PC's and configure the default gateway.

Example

commands:-

Enable

Configure terminal

IP address 192.168.1.1. 255.255.255.224

no shutdown

exit

interface FastEthernet 0/1

IP address 192.168.2.1 255.255.255.224

no shutdown

exit

6] Configure IP address, subnet mask and default gateway on each PC.

7] check the connection b/w PCs using ping command.

8] Send a simple PDU b/w PC's in diff subnet.

Device	IP address	Subnet mask	Gateway
Router	192.168.1.1	255.255.255.224	—
PC1	192.168.1.11	255.255.255.224	192.168.1.1

Result:-

Thus, a subnetting was successfully implemented in Cisco Packet Tracer.

Ex.No:10

Interconnecting with routers in Cisco packet Tracer simulator

Aim:-

To design and configure a simple internetwork using a router and establish communication b/w two diff network.

Requirement:-

Cisco Packet Tracer

1 Router

2 PCs

Copper straight through Cables.

IP configuration:-

Device	IP address	Subnet Mask	Gateway.
Router Fa0/0	192.168.10.1	255.255.255.0	-
Router Fa0/1	192.168.20.1	255.255.255.0	-
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1

Procedure:-

1] Open Cisco Packet tracer.

2] place one router and two PC's in workspace.

3] Connect PC0 to Router 1 Fast Ethernet 0/0 using copper straight through

4] Connect PC1 to Router 1 Fast Ethernet 0/1 using copper straight through cable.

5] open router CLI and enter.

6] Configure PC0:

IP: 192.168.10.2

Subnet mask: 255.255.255.0

Default Gateway: 192.168.10.1

Configure PC1:

IP: 192.168.20.2

Subnet Mask: 255.255.255.0

Default Gateway: 192.168.20.1

7] open command prompt PC0 and type.

ping 192.168.20.2

8] observe the reply packet.

9] Send a simple PDU from PC0 to PC1 and observe the packet transmission.

Result:-

Thus the interconnected network Successfully designed and configured using a router in CISCO Packet Tracer.

Ex.No:11

Routing at Network Layer

Aim:-

To simulate static routing and RIP using CISCO Packet Tracer.

A] Static Routing

It is a method where router are manually added to the router routing table. It is useful for controlling path taken by packet b/w diff network.

Procedure:-

1] open ~~circuit~~ CISCO Packet Tracer.

2] place required routers, switch and Pc in workspace.

3] Connect the device using suitable cables.

4] Assign IP address to all PC's and router interfaces.

5] open the router CLI

6] Enter the foll command to configure static route.

7] Use foll command to check routing table
show Ip route.

8] Use ping command to check connectivity b/w Pc's.

B] RIP Routing :-

It is a dynamic routing protocol. It allows routers to automatically exchange info about the network connected to them. RIP select routes based mainly on no. of hops.

Procedure :-

1] Create network with three routers and PC's shown in packet Tracer topology.
2] Assign IP addresses to PC's and router interfaces.
3] configure the router interface using
4] Enable RIP on router.
5] Repeat RIP configuration for other routers with their directly connected networks.
6] Allow some time for routers to exchange routing information.
7] Check the routing table using show IP route.
8] Test communication using ping <Destinations IP
9] send simple PDU from source PC to destination PC and observe packet path.

Result :-

Thus static routing and RIP were successfully simulated using CISCO Packet Tracer.

Ex-No:-12

End to End Communication at transport layer

Aim:-

To implement an echo client server using TCP/UDP socket and implement a chat client server using socket programming.

a) Echo client server using TCP/UDP:-

It is an endpoint used for communication b/w two device. In an echo program, the client sends a message to server and server sends the same message back to client.

Procedure:-

1] Create server programme and client program
2] Import the required socket library.
3] In server, creates a socket and bind it to an IP address and port no.
4] Make the server listen to the client request
5] Start client and connect it to server.
6] Enter msg from client and sent msg to server.
7] The server receives the msg and sends same msg back.
8] The client receives and display the echoed msg.
9] close socket connection.

b) client server using socket program :-

1. create server and client program using Socket programming
2. Start server and bind it to an IP address and Port
3. Make server wait for client connection.
4. Start client and connect it to server.
5. Enter msg in client and send msg to server.
6. Server receives and display msg.
7. server sends a reply back to client.
8. continue sending and receiving msg to perform a chat
9. Continue sending and receiving msg to perform a chat.
10. close the connection.

Result :-

The end to end Communication was successfully implemented using TCP/UDP Sockets.

Date: 13

Implement the ping program.

Aim:-

To implement a simple ping program to check the connectivity b/w two network devices.

Theory:-

It is a network utility used to check whether a destination device is reachable through a network.

Requirements:-

Computer
Python / suitable programming environment
Network connection.

Program Procedure:-

1. Create a new program for implementation ping operation.
2. Import the required socket / network library.
3. Get IP address or hostname of destination.
4. Create a socket for communication.
5. Send packet to destination.
6. Wait for destination.
7. Display the result and close the connection.

Algorithm :-

1. Start
2. Enter destination IP address.
3. create a socket
4. send a ping request.
5. wait for response
6. If response is received
7. Otherwise display Host is unreachable
8. Stop.

Result :-

Thus the ping program was Successfully implemented.

Ex.No:14 Raw sockets, packet sniffing.

Aim:-

To write a program using Raw sockets to implement packet sniffing.

Theory:-

It is a program used to capture and examine network packets travelling through a network interface.

Requirement:-

Computer
Python
Raw socket support
Network Interface.
Administrator.

Algorithm:-

1] Start the program.
2] Create RAW socket
3] Find the Sockets
4] Wait for incoming network Packets
5] Capture the Packet.
6] Extract Packet info
7] Display Packet Detail.

Procedure :-

1] Open Python IDE.

2] create a Python program.

3] import socket module

4] create RAW socket

5] Use recvfrom() to receive packets

6] Read captured packet data.

7] Display packet info.

Result :-

Thus a RAW socket based packet filling program was implemented successfully.